ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name:**
- **Chinese Name:**
- **Email:**
- **Student ID:**
- **University:**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer: There are 594 documents. Filtering by category lets mining algorithms focus on a specific domain. This helps identify topic-specific patterns rather than patterns that come from mixing unrelated subjects.

AI used for this answer: Yes

If yes, how: Explaining the concept, rewriting the answer to sound better

External references (links): scikit-learn: Newsgroups categories

---

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: Dropping would discard 40% of the dataset. This would cause massiv information/data loss and other problems like sampling bias if the missing labels are not randomly distributed.

An alternative is to retain the unlabeled documents for unsupervised learning, like clustering.

AI used for this answer: Yes

If yes, how: Explaining the concept, rewriting the answer to sound better

External references (links): scikit-learn: Clustering

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer: The proportion of comp.graphics increased from 25.88% in the original dataset to 27.80% in the sample, while soc.religion.christian decreased from 26.54% to 24.80%. Simple random sampling does not guarantee that the category proportions in the sample will exactly match those in the original dataset. In imbalanced datasets, minority classes may become underrepresented or even disappear.

AI used for this answer: Yes

If yes, how: Explaining the concept, rewriting the answer to sound better

External references (links): Inria: Stratification

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer: The total number of features increases by 306 and the sparsity decreases from 99,6772% to 99,5470%. The reason for the decrease is that common words appear in these documents and that results in more non zero entries, which reduces the sparsity.

AI used for this answer: Yes

If yes, how: Explaining the concept

External references (links): scikit-learn: Text feature extraction

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer: The higher the threshold, the fewer features remain, it went down from 35.482 to 1.491. Words that are rare and almost never appear in documents have variance close to zero. These vary little between documents and provide almost non useful information for distinguishing them.

Threshold	Number of terms kept (keep == True)
0.0	35.482
0.01	5.089
0.05	1.491

AI used for this answer: Yes

If yes, how: Explaining the concept

External references (links): scikit-learn: Low-variance feature selection

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer: The top correlated words are not generic stop words, because they are already removed during vectorization. Pearson correlation measures the strength of a linear relationship. Spearman correlation measures the strength of a monotonic relationship using the ranks of the values. This can happen when a word is associated with a topic, but the word count is high in a few documents.

AI used for this answer: Yes

If yes, how: Explaining the concept, rewriting the answer to sound better

External references (links): SciPy: Pearson correlation; SciPy: Spearman correlation

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

The number of discovered patterns increases from 710 to 9.547. The runtime also increases from 0.061 to 1.196 seconds

Answer (2) — Top-K k vs MaxFPGrowth minSup:

In Top-K FAE, the parameter k determines how many of the most occurring patterns are selected. In MaxFPGrowth, minSup defines the minimum frequency required for a pattern to be considered frequent.

Answer (3) — Maximal vs standard frequent patterns:

Maximal frequent patterns keep things simpler by leaving out smaller subsets of larger patterns. This cuts down on a lot of the repetition you usually get with standard pattern mining.

AI used for this answer: Yes

If yes, how: Explaining the concept, rewriting the answer to sound better

External references (links): PAMI: MaxFPGrowth

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer: The X_counts matrix has 35,482 features, but a scatter plot only has two axes. The high dimensional data cannot be visualized directly in a standard 2D scatter plot. Dimensionality reduction algorithms, such as PCA, t-SNE or UMAP, can reduce the data to two dimensions and make it possible to be visualized.

AI used for this answer: No

If yes, how:

External references (links):

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer: In the plots, UMAP shows better separation between the document categories, while PCA shows strong overlap. The t-SNE plot also contains overlapping categories. PCA is a linear method that keeps as much of the variation in the data as possible but it cannot capture non linear relationships as effectively as t-SNE or UMAP. Distances between clusters in a t-SNE plot should not be interpreted as exact measures of similarity because t-SNE mainly preserves local relationships.

AI used for this answer: Yes

If yes, how: Explaining the concept, rewriting the answer to sound better

External references (links): Distill: How to Use t-SNE Effectively

---

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer: Four binary columns are created, one for each category. The codes 0, 1, 2 and 3 imply that the categories have a meaningful order and numerical distances. One-hot binarization gives each category its own binary column. A value of 1 indicates membership in that category, while 0 indicates non-membership.

AI used for this answer: Yes

If yes, how: Explaining the concept, rewriting the answer to sound better

External references (links): scikit-learn: Encoding categorical features

---